Secure Document Sharing System

A secure Flask application for confidential file storage, role-based sharing, and full audit visibility. The deck focuses on architecture, controls, evidence, and trade-offs.

Encrypted at rest

Explicit sharing

78 tests passed

Security posture

Confidential documents stay encrypted at rest.

Uploaded payloads are encrypted before disk write, stored as .bin files, and and verified with HMAC during decryption.

Collaboration and audit

Sharing is explicit, role-scoped, and traceable.

System roles and document roles define who can act, while authentication, upload validation, session lifecycle, and document activity are all recorded to structured logs.

Liam Hughes (lph30) · **Three Peng (sp2269)**

Security Architecture

The system keeps security decisions explicit: middleware gates the request, authorization is layered, and file-based storage is treated as a sensitive trust boundary.

Layer

Client Tier

Browser UI
HTML/CSS/JS
Session token in HttpOnly cookie

Layer

Application Tier

Flask app
app.py, auth.py, sessions.py, access_control.py, documents.py, security.py, storage.py, logging_utils.py

Layer

Persistence Tier

JSON stores, encrypted .bin payloads, security logs, access logs, secret.key

Middleware Sequence

Every request passes four gates.

- HTTPS enforcement
- Session resolution
- CSRF validation
- Security headers

Secure Request and Data Flows

Instead of adding security after the fact, controls are embedded directly in the main user journeys: register or log in, upload, share, and download.

Flow 1

Register / Login

Validate input
Hash with bcrypt cost 12
Lock after 5 failures for 15 min
Rate limit: 10 attempts per IP per min
Create server-side session

Flow 2

Upload

Require authenticated user
Validate extension, MIME, signature, EICAR
Encrypt before disk write
Store SHA-256 hash
Log upload event

Flow 3

Share

Owner or admin manages shares
Resolve document role from shares.json
Support viewer or editor access
Audit share changes
Downgrade editor on return-to-owner

Flow 4

Download

Require auth and document role
Prevent path traversal with safe_file_path()
Decrypt with HMAC verification
Serve version-aware payload
Append audit event

Design principle: validate, authorize, encrypt, and log.

Authentication, Authorization, and Session Security

These controls protect identity, permissions, and session state. Together they prevent the most common account and access failures in a document-sharing system.

Authentication

Credential handling is deliberately strict.

Username validation
Password minimum: 12 chars
Upper, lower, digit, special
bcrypt with gensalt(rounds=12)
Duplicate username and email blocked
Authentication events logged

Authorization

Access control is layered.

System RBAC: admin, user, guest
Document roles: owner, editor, viewer
403 on unauthorized operations
Share management limited to owner or admin
Unknown roles default to guest

Session Security

Sessions stay server-side and short-lived.

secrets.token_urlsafe(32)
Per-session CSRF token
HttpOnly, Secure, SameSite=Strict
Sliding 30-minute timeout
Concurrent session eviction on new login
Logout destroys session record

System roles

admin | user | guest

Document roles

owner | editor | viewer

Data Protection and App Defense

Security is spread across storage, transport, validation, headers, and observability. The goal is not one perfect control, but multiple controls that reinforce each other.

Encryption

Data protection starts before storage.

At rest
Fernet (AES-128-CBC + HMAC-SHA256)
secret.key stored outside data/
SHA-256 plaintext hash tracked in metadata

In transit
HTTPS enforced outside development
HSTS added on secure responses

Key management
Single Fernet key today
Rotation documented as future work

Input Defense

The upload and render surface is heavily checked.

Extension whitelist, MIME validation, magic-byte signature checks, and EICAR scanning stop malformed or suspicious uploads.

safe_file_path() blocks traversal; Jinja2 auto-escaping and CSP reduce XSS exposure.

Monitoring

Headers and logging make attacks visible.

7 security headers are applied on every response.
29 security-relevant event types are logged.
logs/security.log, logs/access.log, and audit.json provide evidence for login, authorization, upload validation, and document activity.

7
required headers

29
logged event types

4
upload validation layers

Threat Model and Testing Evidence

The system was designed around explicit threats and then validated with targeted tests. That gives the project a defensible story beyond just claiming that controls exist.

17
STRIDE threats modeled

78
passing automated tests

5
major attack scenarios

4
core evidence areas

Threat Focus

Representative risks

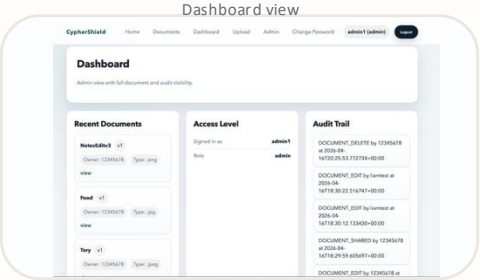
- Brute-force and credential-stuffing on login
- Session hijacking via token theft or XSS
- Path traversal to read arbitrary files
- Malicious file upload disguised as a valid type
- Horizontal privilege escalation on document access

Validation Evidence

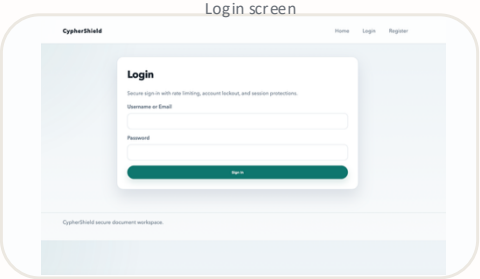
What we verified

- Authentication: lockout, rate limits
- RBAC: roles and admin-only actions
- Documents: upload, share, delete, preview
- Sessions: timeout, cookies, 403, XSS

Dashboard view



Login screen



Challenges, Lessons, and Next Steps

The current design is strong for a teaching project, but it also documents where the simple architecture becomes a limitation. That honesty makes the presentation stronger.

Current Limitations

What remains imperfect today

EICAR-only malware scanning
Single Fernet key with no automated rotation
JSON storage has concurrency limits
CSP still includes 'unsafe-inline'
No secure forgot-password reset flow
Unsigned logs can be modified by a root-level attacker

Future Improvements

Next Steps

Integrate ClamAV or another third-party malware scanning API
Support key rotation with versioned key IDs
Replace JSON storage with SQLite or PostgreSQL
Adopt CSP nonces and remove inline script dependence
Add secure password reset with email token and MFA
Forward logs to tamper-evident centralized storage